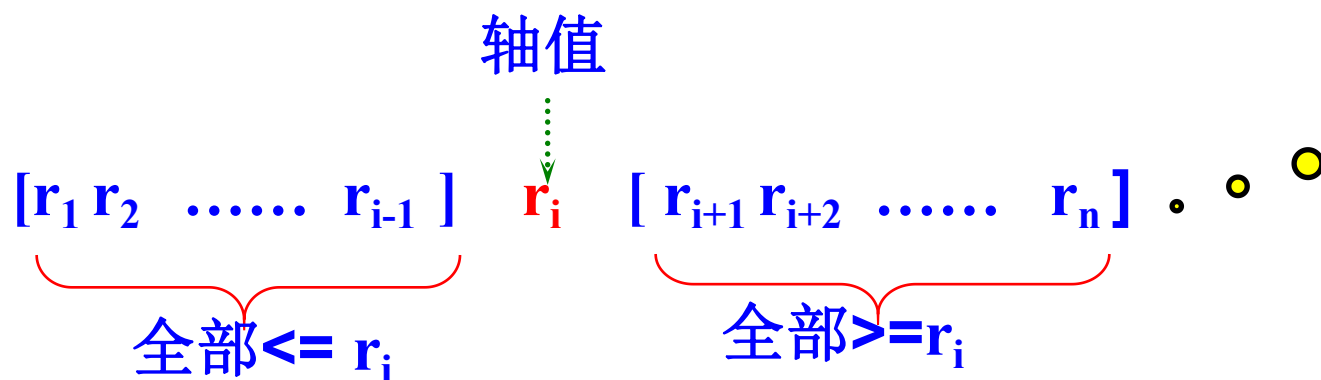


快速排序（选学）

能迅速地对乱序作大幅度调整，特别适用于杂乱无章的数据集合的排序。

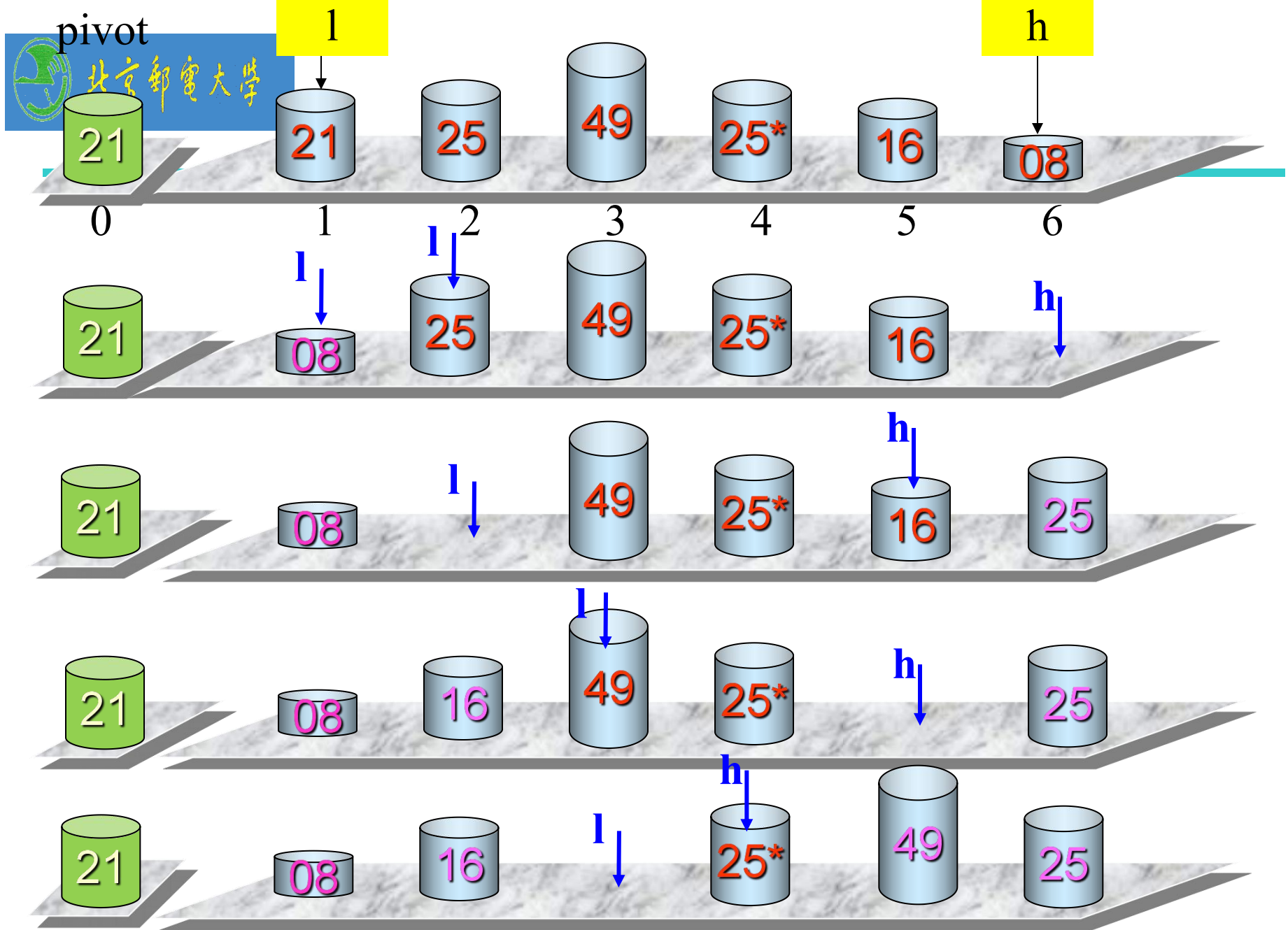
◆ 基本思想：

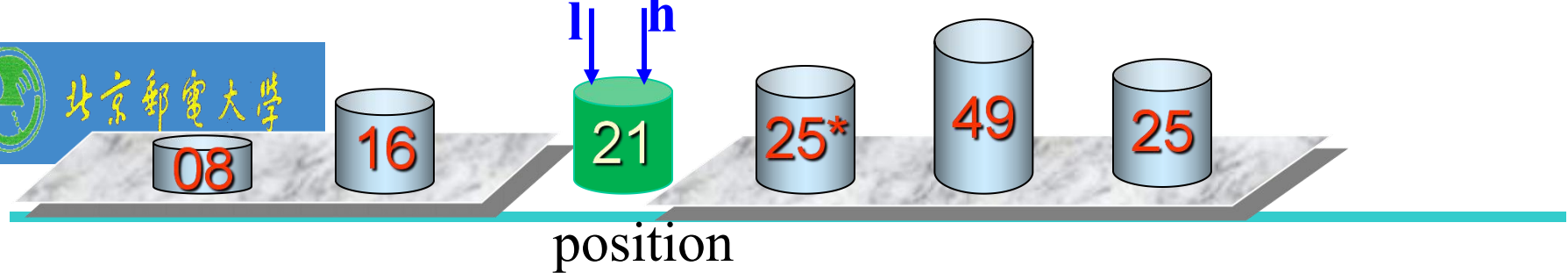
1、选择一个记录作为枢轴值，将记录分成2部分。



递归

2、分别对这两部分重复上述过程。直到区间内只有一个元素。





• 算法

① 初始化

取第一个记录 $r[low]$ 作为基准，保存在 $[0]$ 位置

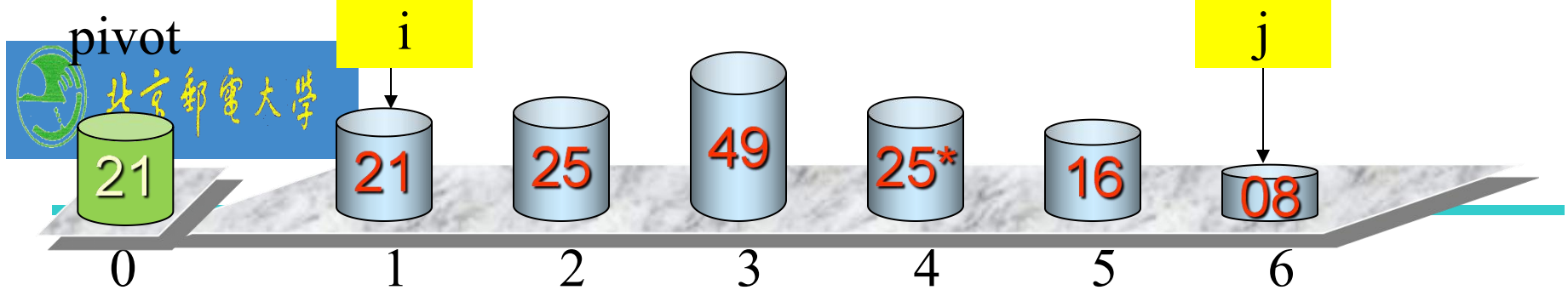
② 右侧扫描

从后向前找到第一个比基准小的记录，移至位置 low ， $low++$

③ 左侧扫描

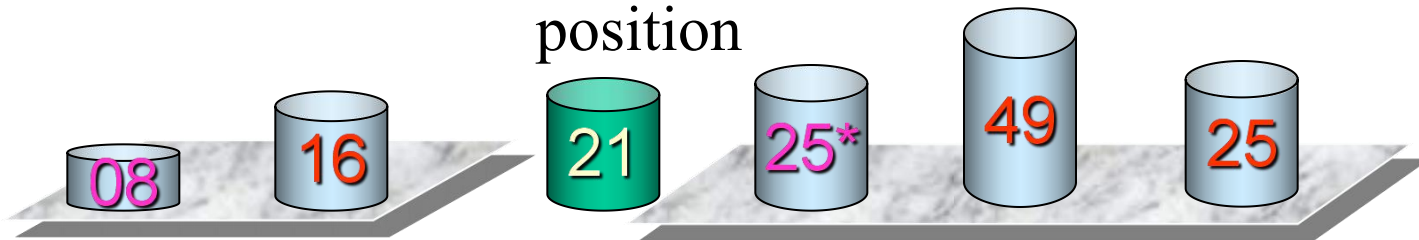
从前到后找到第一个比基准大的记录，移至位置 $high$ ， $high--$

反复执行②③，直到 $low=high$ 结束，将 $r[0]$ 移至 $r[low]$

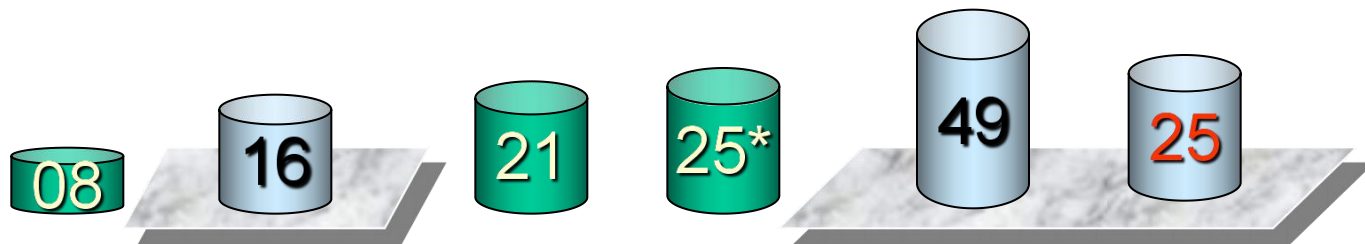


position

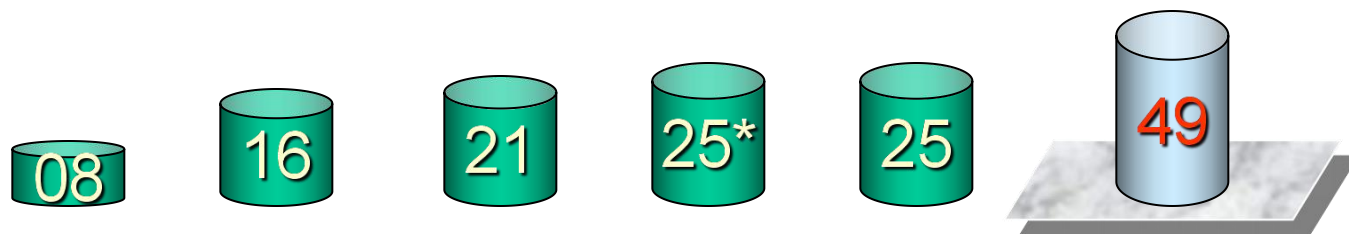
第一趟



第二趟



第三趟



快速排序的递归算法

```
void quick_sort (int a[], int low, int high)
{
    int pivotloc;
    if ( low < high )
    {
        pivotloc = partition(a, low, high ) ;
        quick_sort (a, low, pivotloc-1) ;
        quick_sort (a, pivotloc+1, high ) ;
    }
} //递归的快速排序算法
```

快速排序的核心算法

```
int partition (int a[], int low, int high)
{ int pivotkey = a[low];
  while (low < high ){
    while (low<high && a[high] >=pivotkey)
high--;
    a[low] =a[high];
    while (low<high && a[low]<=pivotkey )
low++;
    a[high] = a[low];
  }
  a[low]=pivotkey;
  return low;
} // Partition
```

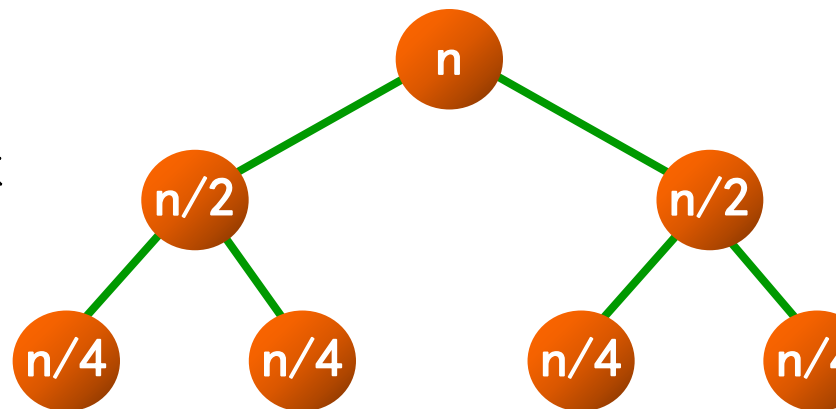
快速排序算法评价

- 最好情况

每次划分的左侧序列和右侧序列的长度相同。表长为 n 的序列可划分 $\log_2 n$ 层。

定位一个记录要对整个待划分序列扫描一遍，所需时间 $O(n)$ 。

总的时间复杂度： $O(n \log_2 n)$ 。



- 最坏情况

若待排记录的初始状态有序（正、逆）时，快速排序将蜕化为起泡排序，其时间复杂度为 $O(n^2)$ 。

- 空间复杂度：递归深度。最好 $O(\log_2 n)$ ，最坏情况 $O(n)$ 。

归并排序（选学）

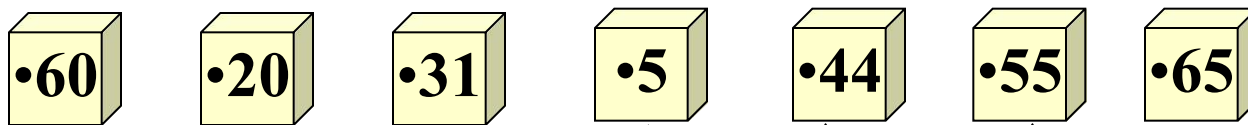
- **归并：**是指将若干个已排好序的部分合并成一个有序的部分。
- **归并分类：**
 - 二路归并、三路归并、多路归并
- **归并排序的实现有两种方法：**
 - 非递归方法和递归方法。



二路归并排序

•初始序列: •60 20 31 5 44 55 65

•初始:



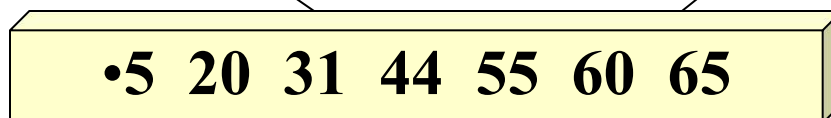
•一趟:



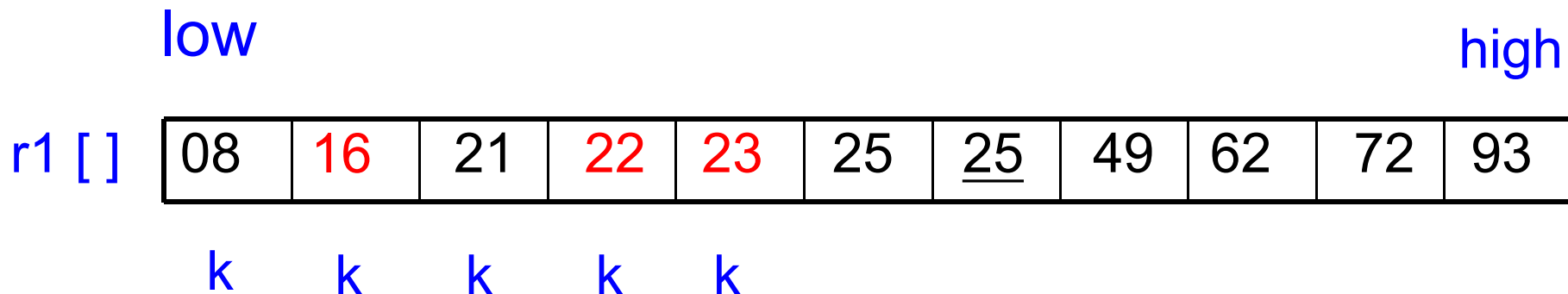
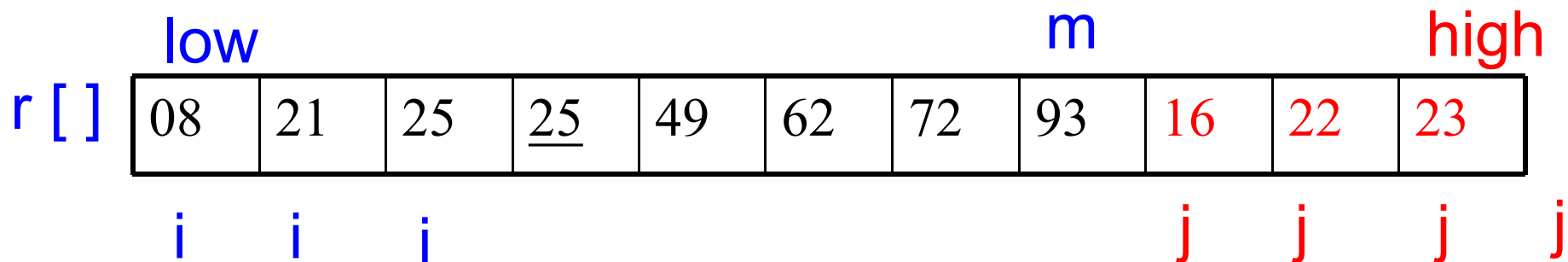
•二趟:



•三趟:



合并：将两个有序表合并为一个有序表





归并排序的递归算法

```
void merge_sort(int L[],int low,int high)
{
    int mid;
    if (low < high){
        mid=(low+high)/2;
        merge_sort(L,low,mid);
        merge_sort(L,mid+1,high);
        merge(L,low,mid,high);
    }
}
```

归并排序的核心算法 (1)

```
void merge(int L[], int low, int mid, int high){
    int k, begin1, begin2, end1, end2;
    begin1 = low;      end1 = mid;
    begin2 = mid + 1;  end2 = high;
    //int temp[high-low+1];
    int *temp = (int *) malloc((high - low + 1) *
    sizeof(int));

    //2.将左右两部分合并排序
    for(k = 0; begin1 <= end1 && begin2 <= end2;
    k++){
        if(L[begin1] <= L[begin2])
            temp[k] = L[begin1++];
        else temp[k] = L[begin2++];
    }
```

归并排序的核心算法（2）

```
if(begin1 <= end1) //左剩
    memcpy(temp+k, L+begin1, (end1-
begin1+1)*sizeof(int));
else //右剩
    memcpy(temp+k, L+begin2, (end2-
begin2+1)*sizeof(int));

//3. 排序后复制到原数组
    memcpy(L+low, temp, (high-
low+1)*sizeof(int));
    free(temp); //释放空间
}
```

归并排序的非递归算法

```
void mergesort ( int L[],int last ) {  
    int len,low,mid,high,i;  
    for (len=1 ; len < last; len*=2 )  
    {  
        low = 1;  
        while (low + len <= L->last)  
        {  
            mid= low+len-1; high =  
mid+len;  
            if(high>last) high=last;  
            merge(L,low,mid,high);  
            low=high+1;  
        }  
    }  
}
```

5. 归并排序的性能分析

- 时间性能

归并排序需要 $\log_2 n$ 趟，每趟需要把结果存在 $r[1 \sim n]$ 中，需要 $O(n)$

时间复杂度： $O(n \log_2 n)$

- 空间性能

空间复杂度： $O(n)$

- 归并排序是稳定的排序方法

- 较少用于内部排序

- 缺点：合并数组需要额外存储空间，解决方法是链表；

- 优点：效率高；用迭代取代递归，小部分使用插入排序，效率会更高。



各种排序方法的综合比较

排序方法	平均时间	最坏情况	最好情况	辅助空间	稳定性
插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	√
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	×
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	√
快速排序	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(\log n)$	×
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	√

- 快速排序是目前基于比较的内部排序中最好的方法
- 关键字随机分布时，快速排序的平均时间最短，堆排序次之，但后者所需的辅助空间少
- 当 n 较小时如($n < 50$)，可采用直接插入或简单选择排序，前者是稳定排序，但后者通常记录移动次数少于前者
- 当 n 较大时，应采用时间复杂度为 $O(n \lg n)$ 的排序方法(主要为快速排序和堆排序)或者基数排序的方法，但后者对关键字的结构有一定要求
- 当 n 较大时，为避免顺序存储时大量移动记录的时间开销，可考虑用链表作为存储结构(如插入排序、归并排序、基数排序)

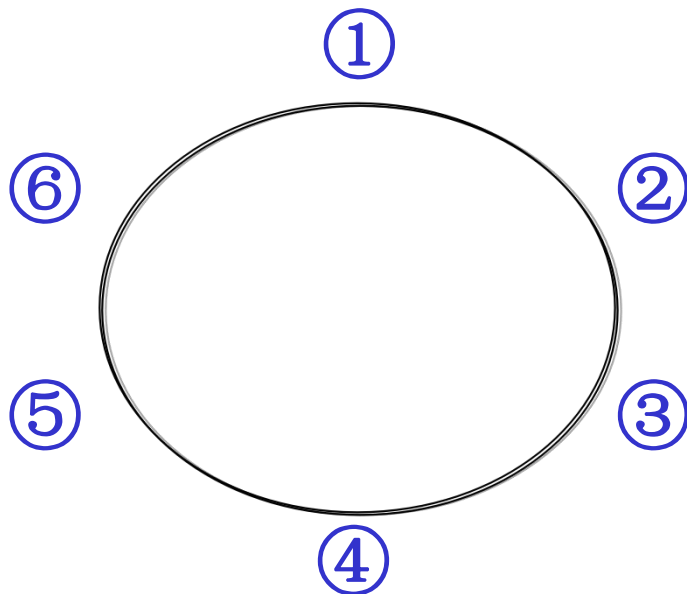
约瑟夫环问题：

一圈共有 N 个人，开始报数，报到 M 的人自杀，然后重新开始报数，问最后自杀的人是谁？

约瑟夫环：猴子选大王

问题描述：

猴子选大王：有 N 只猴子围成一圈，每只猴子从1到 N 依次编号。打算从中选出一个大王，经过协商，决定出选大王的规则：从第一个猴子开始循环报数，数到 M 的猴子出圈，最后剩下下来的就是大王。



若数到4的猴子出圈，
则出圈的猴子编号
依次是：4、2、1、
3、6

编号为5的猴子即为
大王。

- **猴子可以被抽象为序号。**
- **猴子具有相邻关系（线性）**，因此可以用一个一维数组来存储猴子的序号，存储位置的相邻体现了猴子之间的相邻。

数组a

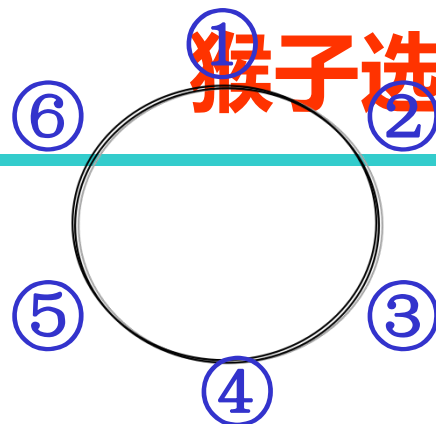
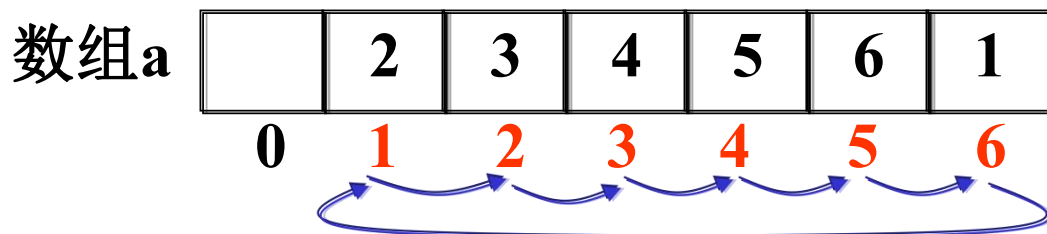
1	2	3	4	5	6
---	---	---	---	---	---

数组a

1	2	3	5	6	
---	---	---	---	---	--

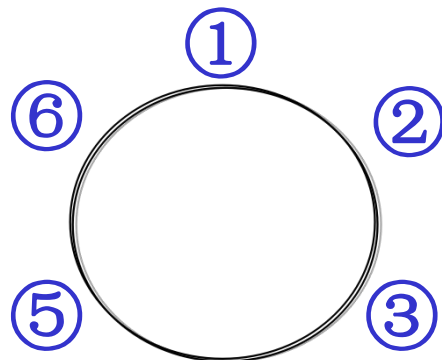
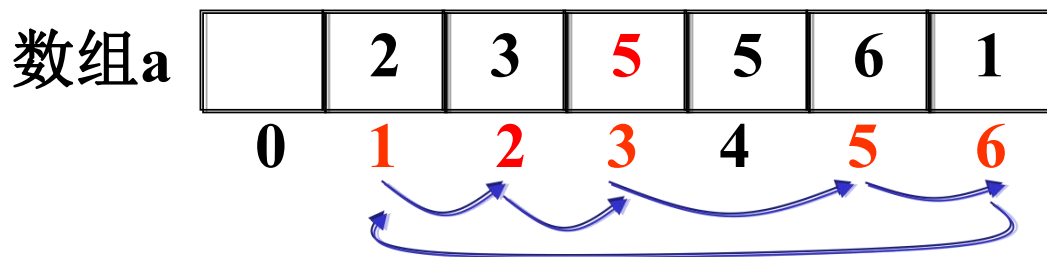
- 当猴子4出圈后，从数组中删除对应的序号
- 这种存储方式可以解决问题，但是删除操作涉及数组元素的批量移动，算法不够高效。
- 能否在一个猴子出圈的时候不用删除一个数组元素？

猴子选大王

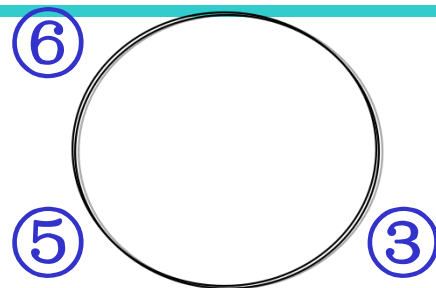
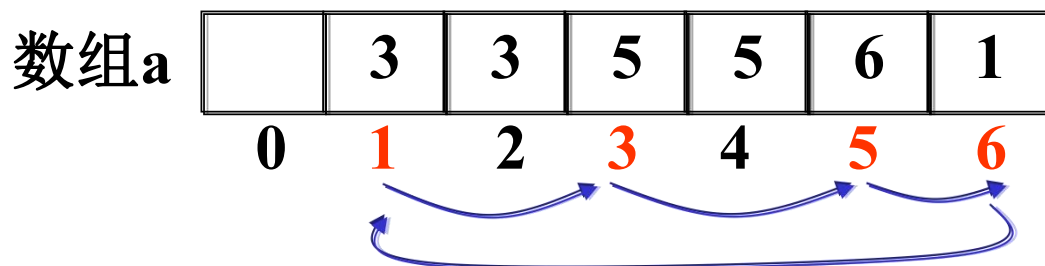


数组下标*i*代表猴子编号； $a[i]$ 存储编号为*i*的猴子的下一个猴子的编号（应用指针的原理）。例如：下标为1的数组元素存放值2，表示编号为1的猴子的下一个猴子是2。

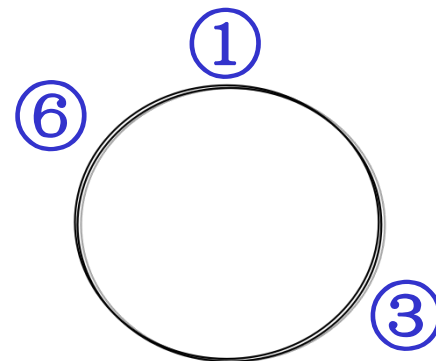
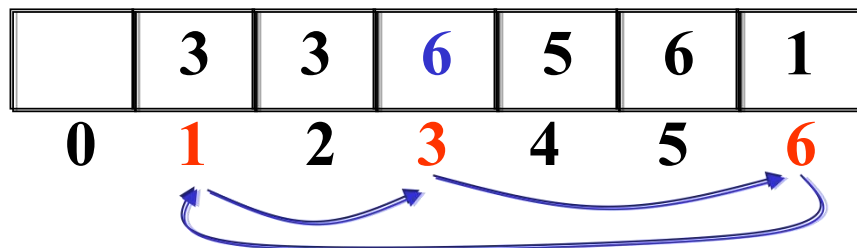
当编号为4的猴子出圈后，4号后面猴子的编号保存到4号前面猴子处： $a[3]=a[4]$



猴子选大王-出圈



- 假设5号猴子要出圈，则需要把5号后面猴子的编号保存到5号前面猴子处： $a[3]=a[5]$

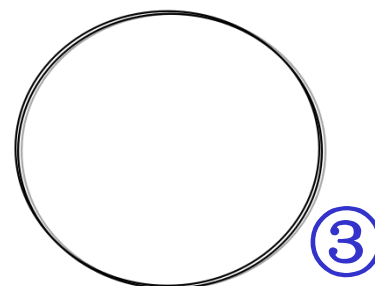


更一般的，编号为i的猴子出圈：

$a[prei]=a[i]$ ；其中prei是圈中第i号猴子前面猴子的编号

- 如何判断只剩一只猴子：

	X	X	3	X	X	X
0	1	2	3	4	5	6



方法1：若当前访问的序号为 i 的猴子是最后一个猴子，则 $a[i] == i$ 。如图，假设当前只剩3号猴子，则 $a[3] == 3$

方法2：设置一个计数器用来计算出圈猴子的数量，当出圈数量为猴子总数-1时，表示只剩一只猴子

count=0;i=猴子总数

⑥ !a[i]=i

① count++

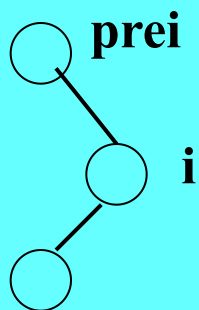
④ prei=i;i=a[i]

② count%4==0

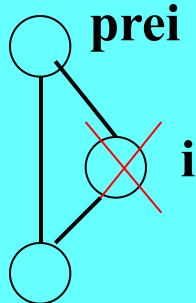
③ a[prei]=a[i]

⑤ i=prei(当编号为i的猴子出圈后,为了能通过i=a[i]操作数到下一个猴子,必须执行此操作)

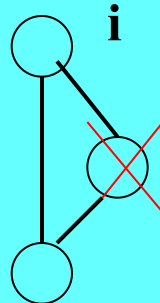
无



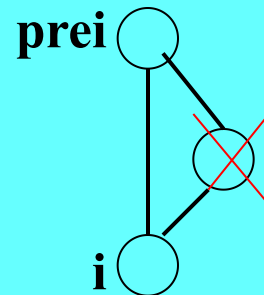
出圈: $a[prei]=a[i]$
 $i=prei$



下次报数前要满足
的状态



执行下次报数
 $prei=i;i=a[i]$ 后




```
int main(void)
{
    int i,m,n , int a[40];
    printf("输入猴子总数: ");
    scanf("%d",&n);
    printf("输入间隔数: ");
    scanf("%d",&m);

    for(i=1;i<=n-1;i++)
        a[i]=i+1;
    a[n]=1;

    selectKing(a,n,m);
    return 0;
}
```

```
void selectKing(int a[],int n,int m)
{
    int count,prei,i;
    count=0;
    i=n;
    while(a[i]!=i){
        count++; //报数
        prei=i;
        i=a[i];
        if(count%m==0){//出圈
            a[prei]=a[i];
            printf("%d号猴子出圈\n",i);
            i=prei; //下次循环准备
        }
    }
    printf("%d号是大王",i);
}
```

约瑟夫环算法-1创建环

```
int
create_joseph_cycle(int
a[],int n){
    int i;
    for(i=1;i<=n-1;i++)
        a[i]=i+1;
    a[n]=1;
    return n;
}
```

```
int main(){
    int a[100]={0},n,m;
    printf("输入人数n: ");
    scanf("%d",&n);
    create_joseph_cycle(a,
n);
    printf("输入报数m: ");
    scanf("%d",&m);
    josephCycle(a,n,m);
    return 0;
}
```

约瑟夫环算法2-报数出圈

```
void josephCycle(int a[],int n,int m){
    int count,prei,i;
    count=0;    i=n;//预备从1号开始的前编号
    while (a[i]!=i){
        count++; //报数
        prei=i;
        i=a[i];//找下一个
        if(count%m==0){//出圈
            a[prei]=a[i];
            printf("%d号出圈\n",i);
            i=prei;//出圈后返回前结点
        }
    }
    printf("The last is No.%d !",i);
}
```

